مكتب

Reference Notes

Week 7: Introduction to Lists

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How do different data structures help us organize and manipulate information? This week, we begin exploring lists - Python's most versatile way to store collections of data. Unlike working with individual variables, lists let us manage multiple related values together, opening up entirely new programming possibilities!

Prerequisites

Before starting this week, you should be comfortable with:

- Variables and basic data types from Module 1
- For loops for iterating through sequences
- Basic problem decomposition
- Using `input()` and `print()`

1.2 What You Already Know

From Module 1, you've worked with individual variables that store single values. You've used for loops with `range()` to repeat actions. You understand how to get input from users and display output. Now we'll build on this foundation to work with collections of data all at once!

1.3 What You'll Be Able to Do

By the end of this week, you'll create programs that can:

- Store multiple related values in a single list variable
- Add, remove, and modify items in your lists
- Process entire collections of data with loops
- Build interactive programs like high score trackers
- Choose when lists are the right tool for your problem

2 VIDEO 1: Understanding Lists

2.1 Understanding Lists

Imagine you're creating a game that tracks the top 10 high scores. Using individual variables, you'd need score1, score2, score3... all the way to score10. That's tedious and inflexible! Lists solve this problem by letting us store multiple values in a single, organized container. A list is like a numbered shelf where each slot can hold any type of data.

2.2 Your First List

Let's create our first list to see how much simpler our code becomes. We'll make a list of test scores and see how Python organizes them with automatic numbering starting from 0.

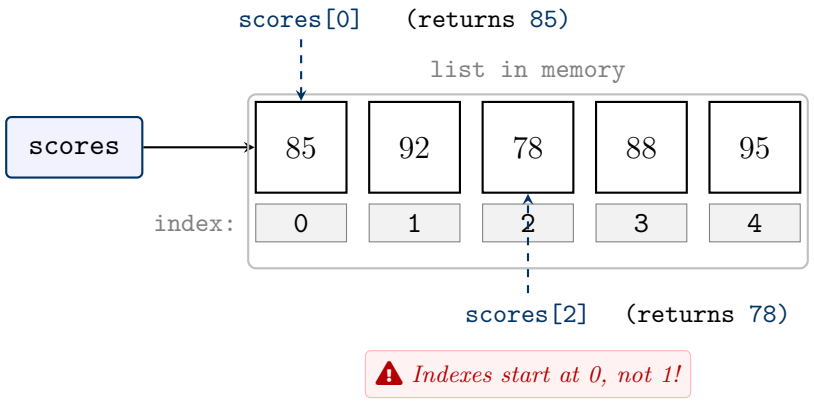
```
# Individual variables (the old way)
score1 = 85
score2 = 92
score3 = 78

# Using a list (the better way!)
scores = [85, 92, 78, 88, 95]
print(scores)
print("First score:", scores[0])
print("Third score:", scores[2])
```

Notice how we use square brackets to create a list and access items by their position (called an index). Python starts counting from 0, so the first item is at position 0.

2.3 Visualizing List Structure

Understanding how lists are organized in memory helps us work with them effectively. Each item has a position number (index) that we use to access it, similar to house numbers on a street.



This diagram shows why `scores[0]` gives us 85 (the first item) and `scores[2]` gives us 78 (the third item). The index is always one less than the item's position in our counting.

2.4 Lists Can Hold Different Types

One powerful feature of Python lists is their flexibility - they can store different types of data in the same list. This makes them perfect for representing complex real-world information.

```python
# A list with different data types
student_info = ["Ali", 13, 8.5, True]
print(student_info)

# Lists can even contain other lists!
classroom = [
    ["Ali", 85],
    ["Sara", 92],
    ["Ahmed", 88]
]
print(classroom[1])  # ['Sara', 92]
```

Here we see lists can contain strings, integers, floats, booleans, and even other lists! This flexibility makes lists incredibly useful for organizing related information.

Tip

While lists *can* hold different types, it's usually clearer to keep all items the same type. A list of scores should contain only numbers, a list of names should contain only strings.

Quick Check

What will this code print?

```python
colors = ["red", "blue", "green", "yellow"]
print(colors[1])
print(colors[-1])
```

Hint: Negative indexes count from the end!

3 VIDEO 2: Essential List Methods

3.1 Understanding List Methods

Lists come with built-in methods - special functions that know how to work with list data. These methods let us add items, remove items, find items, and reorganize our lists. Think of them as the list's built-in toolkit that makes common operations easy.

3.2 Adding Items to Lists

The most common list operations involve adding new items. Python gives us two main methods: `append()` adds one item to the end, while `insert()` adds an item at any position we specify.

```python
# Start with an empty shopping list
shopping = []

# Add items one by one
shopping.append("milk")
shopping.append("bread")
shopping.append("eggs")
print(shopping)  # ['milk', 'bread', 'eggs']

# Insert at specific position
shopping.insert(1, "butter")  # Insert at index 1
print(shopping)  # ['milk', 'butter', 'bread', 'eggs']
```

See how `append()` always adds to the end, making our list grow? And `insert()` lets us place items exactly where we want them, shifting other items to make room.

3.3 Removing Items from Lists

Just as we can add items, we need ways to remove them. Python provides three methods depending on what we know about the item we want to remove.

```
fruits = ["apple", "banana", "orange", "banana", "grape"]

# Remove by value (first occurrence)
fruits.remove("banana")
print(fruits)  # ['apple', 'orange', 'banana', 'grape']

# Remove by index
deleted = fruits.pop(2)  # Removes and returns item
print(f"Removed: {deleted}")
print(fruits)  # ['apple', 'orange', 'grape']

# Remove last item
fruits.pop()  # No index means remove last
print(fruits)  # ['apple', 'orange']
```

Notice how `remove()` finds and deletes the first matching value, while `pop()` works with positions and even gives us back the removed item!

3.4 Searching and Counting in Lists

Before we can work with list items, we often need to find them first. Python provides methods to check if items exist, find their position, and count how many times they appear.

```
grades = [85, 92, 85, 78, 92, 85, 90]

# Check if item exists
if 92 in grades:
    print("Someone scored 92!")

# Find position of item
position = grades.index(78)
print(f"78 is at index {position}")

# Count occurrences
count = grades.count(85)
print(f"85 appears {count} times")
```

These searching methods help us answer questions about our data: Is this item there? Where is it? How many are there?

Common Error

Common Mistake: Using `index()` on a value that doesn't exist causes an error! Always check with `in` first:

```
if value in my_list:
    position = my_list.index(value)
```

3.5 Useful List Operations

Python provides additional operations that help us analyze and manipulate our lists efficiently. These work especially well with lists of numbers.

```
scores = [85, 92, 78, 88, 95]

# Get list length
size = len(scores)
print(f"Number of scores: {size}")

# Mathematical operations (for number lists)
total = sum(scores)
highest = max(scores)
lowest = min(scores)

print(f"Total: {total}, Highest: {highest}, Lowest: {lowest}")
print(f"Average: {total/size}")
```

These built-in functions save us from writing loops to calculate common values. They make our code cleaner and easier to understand!

Quick Check

Given the list `numbers = [10, 20, 30, 40]`, what happens after: 1. `numbers.append(50)` 2. `numbers.insert(2, 25)` 3. `numbers.pop(0)`
Trace through each operation mentally!

4 VIDEO 3: Dynamic List Operations

4.1 Understanding Dynamic Lists

The real power of lists comes from their ability to change size during program execution. Unlike arrays in some languages, Python lists can grow and shrink as needed. This dynamic behavior lets us build programs that adapt to user input and changing conditions.

4.2 Building Lists with Loops

Instead of hardcoding list values, we can build lists dynamically using loops. This is perfect for collecting user input or generating data based on calculations.

```
# Collect grades from user
grades = []  # Start empty
num_grades = int(input("How many grades? "))

for i in range(num_grades):
    grade = int(input(f"Enter grade {i+1}: "))
    grades.append(grade)

print(f"You entered: {grades}")
average = sum(grades) / len(grades)
print(f"Average: {average:.1f}")
```

This pattern of starting with an empty list and building it up with `append()` is extremely common. The list grows exactly as large as needed!

4.3 Processing Lists with Loops

Once we have a list, we often need to process each item. Python gives us two elegant ways to iterate through lists, each useful in different situations.

```
names = ["Ali", "Sara", "Ahmed", "Fatima"]

# Method 1: Direct iteration (when you need values)
print("Class list:")
for name in names:
    print(f"- {name}")

# Method 2: Index iteration (when you need positions)
print("\nNumbered list:")
for i in range(len(names)):
    print(f"{i+1}. {names[i]}")
```

The first method is simpler when you just need the values. The second method is necessary when you need to know positions or modify items.

4.4 Modifying Lists While Processing

We can change list items as we iterate through them, but we need to be careful about how we do it. Modifying by index is safe, but changing the list's size during iteration requires special techniques.

```
# Safe: Modifying values by index
prices = [100, 200, 150, 300]
print(f"Original: {prices}")

# Apply 10% discount to all items
for i in range(len(prices)):
    prices[i] = prices[i] * 0.9

print(f"After discount: {prices}")
```

This works because we're changing values, not adding or removing items. The list size stays the same throughout the loop.

4.5 Filtering and Transforming Lists

A common pattern is creating a new list based on an existing one - either selecting certain items (filtering) or changing all items (transforming). Building a new list is often clearer than modifying the original.

```
numbers = [15, 8, 23, 4, 16, 42, 9]

# Filter: Keep only numbers > 10
big_numbers = []
for num in numbers:
    if num > 10:
        big_numbers.append(num)
print(f"Numbers > 10: {big_numbers}")

# Transform: Square all numbers
squares = []
for num in numbers:
    squares.append(num ** 2)
print(f"Squares: {squares}")
```

This pattern of iterating through one list to build another is fundamental to data processing. It keeps our original data intact while creating exactly what we need.

Rule

When removing items from a list you're iterating over, create a new list instead! Removing items during iteration can cause you to skip elements because the indexes shift.

Quick Check

Write the output of this code:

```
data = [5, 10, 15, 20]
result = []
for x in data:
    if x >= 10:
        result.append(x * 2)
print(result)
```

5 VIDEO 4: Common List Patterns

5.1 Understanding List Patterns

Certain list operations appear so frequently in programming that they've become standard patterns. Recognizing and mastering these patterns will make you a more efficient programmer. Let's explore the most important ones you'll use repeatedly.

5.2 The Accumulator Pattern

The accumulator pattern builds up a result by processing items one at a time. We start with an initial value (often 0 or an empty list) and update it as we go through the list.

```
# Pattern 1: Numeric accumulator
numbers = [10, 25, 30, 15]
total = 0  # Accumulator starts at 0

for num in numbers:
    total = total + num  # Accumulate

print(f"Sum: {total}")

# Pattern 2: List accumulator
words = ["python", "is", "fun"]
long_words = []  # Accumulator starts empty

for word in words:
    if len(word) > 3:
        long_words.append(word)  # Accumulate

print(f"Long words: {long_words}")
```

The accumulator pattern is like a snowball rolling downhill - it starts small and gathers more as it goes. This pattern forms the basis for many algorithms!

5.3 The Find Pattern

Often we need to search for items meeting specific criteria. The find pattern helps us locate items efficiently and handle cases where items might not exist.

```
# Finding the first item that matches
scores = [65, 82, 91, 78, 95, 88]
found_a = False

for score in scores:
    if score >= 90:
        print(f"Found A grade: {score}")
        found_a = True
        break  # Stop after finding first

if not found_a:
    print("No A grades found")

# Finding ALL items that match
high_scores = []
for score in scores:
    if score >= 85:
        high_scores.append(score)

print(f"All high scores: {high_scores}")
```

Notice how we use a flag variable (`found_a`) to track whether we found anything. This prevents errors when nothing matches our criteria.

5.4 The MinMax Pattern

Finding the smallest or largest value in a list is another essential pattern. While Python has `min()` and `max()` functions, understanding the pattern helps with more complex searches.

```
temperatures = [28, 32, 29, 35, 31, 33, 30]

# Find maximum manually
hottest = temperatures[0]  # Start with first

for temp in temperatures:
    if temp > hottest:
        hottest = temp

print(f"Hottest day: {hottest} ° C")

# Find position of maximum
hottest_day = 0
for i in range(len(temperatures)):
    if temperatures[i] > temperatures[hottest_day]:
        hottest_day = i

print(f"Hottest was day {hottest_day + 1}")
```

Starting with the first element as our initial "best" ensures we always have a valid comparison. This pattern extends to finding items with any "best" quality!

Tip

When using these patterns, always handle empty lists! Check `if len(my_list) > 0:` before processing to avoid errors.

Quick Check

Using the accumulator pattern, mentally trace this code:

```
values = [1, 2, 3, 4]
result = 1
for v in values:
    result = result * v
print(result)
```

What does this calculate?

6 VIDEO 5: Summary

6.1 What We Learned This Week

Congratulations! You've unlocked one of Python's most powerful features - lists! This week, you transformed from working with individual variables to managing entire collections of data. You learned how lists organize multiple values with indexes starting at 0, mastered essential methods like `append()`, `remove()`, and `pop()`, and discovered how to build and process lists dynamically with loops. Most importantly, you learned common patterns that appear in countless programs - accumulator, find, and minmax patterns that you'll use again and again.

Landmark Moment

You can now create programs that handle unlimited amounts of data! From managing game high scores to processing survey responses, lists give you the power to build truly dynamic applications. Next week, we'll explore what happens when we copy lists and why understanding list mutability is crucial for avoiding bugs.

7 Quick Reference

Quick Reference

Week 7 Essential Patterns:
Creating Lists:

- Pattern: `my_list = [item1, item2, item3]`
- Empty list: `my_list = []`
- Common use: Starting point for data collection
- Common mistake: Forgetting square brackets

List Methods:

- Pattern: `list.append(item)` - adds to end
- Pattern: `list.insert(index, item)` - adds at position
- Pattern: `list.remove(value)` - removes first match
- Pattern: `list.pop(index)` - removes and returns
- Debug tip: Check if item exists before using `index()`

Processing Lists:

- Pattern: `for item in list:` - when you need values
- Pattern: `for i in range(len(list)):` - when you need indexes
- Common use: Building new lists from existing ones
- Common mistake: Modifying list size while iterating

7.1 Reflection Questions

1. What was the most challenging part about understanding list indexing starting from 0?
2. How do you decide whether to use `append()` or `insert()` when adding items?
3. Which list pattern (accumulator, find, or minmax) makes the most sense to you and why?
4. Can you think of a real program you'd like to build now that you know about lists?
5. How are lists different from the individual variables you used before, and when would you still use individual variables?

Next week: We'll discover why `list2 = list1` doesn't create a copy and learn about list mutability!